

FRONTEND ENGINEERS • UX CONTRIBUTORS

UI MANUAL

Chart, editor, UI shell, indicators, results, store

Open charting PWA — own the axes, swap the engine



CONTENTS

01	UI
02	Charting
03	Editor
04	UI shell
05	Indicators
06	Results and strategy
07	Store

UI

AXIS UI subsystem: chart host, editor, UI shell, indicators, results, and Solid store—presentation without a closed engine.

UI

UI is the AXIS browser presentation layer: everything the operator sees and clicks that is *not* language evaluation. Engines are plugins; AXIS applies their outputs to panes, markers, and drawers.

Abstract

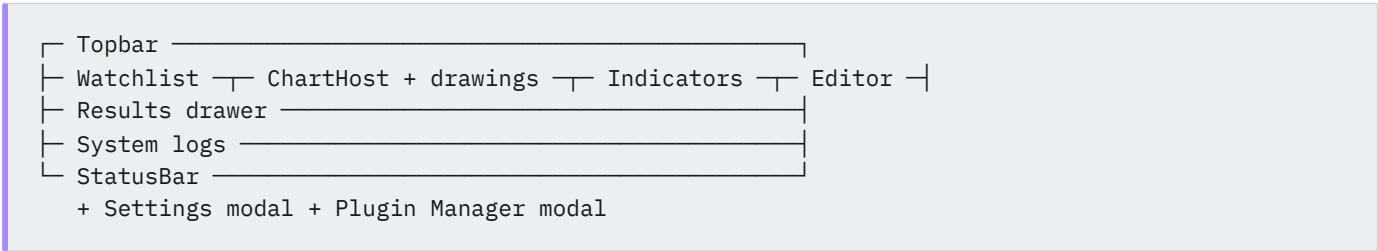
Subsystem	Responsibility
Charting	lightweight-charts panes, drawings, crosshair
Editor	CodeMirror 6 Pine surface, dock/popout
UI shell	Topbar, watchlist, status, manager chrome
Indicators	Run pipeline, overlay apply, indicator list
Results & strategy	Event normalize, trades, export
Store	Solid store, persistence, active plugins

Conceptual model

```
flowchart TB
    SHELL[UI shell] --> STORE[Solid store]
    ED[Editor] --> STORE
    ED -->|Run| IND[indicators/runner]
    IND --> ENG[Engine plugin]
    ENG --> IND
    IND --> CH[Chart host]
    IND --> RES[Results]
    STORE --> CH
    STORE --> SHELL
```

Invariant (AXIS ≠ engine): no Solid component imports PYNE directly; only engine modules talk to Python/HTTP runtimes.

Interface surface (visual map)



Implemented in `frontend/src/app.tsx`.



Stack

Concern	Choice
Framework	SolidJS 1.9
Bundler	Vite 6 + vite-plugin-solid
Charts	lightweight-charts 5
Editor	CodeMirror 6
Icons	lucide-solid
Style	Tailwind 4 + void tokens (data-theme)

Relation to other tracks

- Operators: [End User](#)
- Structure: [Architecture](#)
- Contracts: [Plugins](#)
- Language: [PYNE runtime](#)

See also

- [frontend/LEGACY.md](#) — what is *not* the AXIS product path
- [frontend/TESTING.md](#) — unit/e2e around UI modules



CHARTING

AXIS chart: ChartHost, PaneManager, series factory, drawings, crosshair sync, and trade markers.

CHARTING

Charting is the spatial surface for OHLCV, overlays, and annotations. Implementation is **imperative** under a Solid shell so lightweight-charts owns its DOM subtree.

Abstract

Module	Role
ChartHost.tsx	Solid mount; empty-state overlay; lifecycle
pane-manager.ts	Multi-pane LWC charts, time sync
series-factory.ts	Candle/volume/line series helpers + palette
drawing-layer.ts	User + script drawings
crosshair-sync.ts	Cross-pane crosshair
pine-drawings.ts	Engine drawing objects → layer
manager-access.ts	Process-wide manager/layer getters

Conceptual model

```
flowchart TB
    HOST[ChartHost Solid] --> EL[panesEl div]
    EL --> PM[PaneManager]
    PM --> PRICE[price pane]
    PM --> VOL[volume pane]
    PM --> IND[indicator pane?]
    PM --> EQ[equity pane?]
    PM --> DL[DrawingLayer]
    RUN[runAndApply] --> PM
    RUN --> DL
    BARS[store.bars] --> PM
```

Interface surface

Panes

store.panes: { id, type, height, order, visible, label }.



type	Typical series
price	Candles + overlay lines + markers + drawings
volume	Histogram
indicator	Non-overlay plots
equity	Strategy equity curve

Defaults: price + volume. Indicator/equity created on demand.

Empty / status overlay

When `bars.length === 0`, `ChartHost` shows contextual title/sub from `store.status` (loading, error, running, or “Load data to begin”).

Drawings toolbar

See operator guide [Drawings](#). Tools: cursor, hline, trend, ray, rect, fib, measure, text.

Crosshair & time

`syncTimeScales` / `syncCrosshair` keep multi-pane navigation coherent. `scrollToTime` used from Results trade clicks.

Internals

Solid vs imperative boundary

```
// ChartHost – Solid owns outer shell
// PaneManager only mutates panesEl – never put Solid children inside panesEl
```

On cleanup: destroy drawing layer, dispose manager, clear accessors.

Data path

`setDataToChart(bars)` applies OHLCV; `createEffect` on `store.bars` re-applies if bars change outside load helper.

Run apply (chart side)

From `runAndApply` (indicators):

- `syncOverlayLines` — update existing overlay series in place (avoids destroy → blank → add flash on live re-runs)
- Markers from normalized events (set in place)
- Script drawings atomic replace via `DrawingLayer` (`setScriptDrawings` without empty frame)
- Equity series when strategy report warrants (silent live runs skip hide thrash)

History vs live bars

- Full loads: `loadBars` → `chartDataGen++` → `setDataToChart` + `fitContent`
- Live: `multiplex` → `appendBar` + `manager.appendBar` only (no `fitContent`, no full `setData`)



series-factory

Centralizes series construction options and color palette so Results plot summary and chart stay chromatically related.

Invariants

1. Solid reconciliation must not rewrite pane roots.
2. Script drawings ephemeral per run; user drawings store-backed.
3. Overlay default true unless `meta.overlay === false`.
4. Manager may be null before mount—runner no-ops chart apply safely.

Failure modes

Symptom	Cause
Blank panes after hot reload	Manager disposed; full remount
Overlays stack forever	Old path skipped removeOverlays
Fib/tools ignore clicks	No bars / time scale; wrong tool
Markers at t=0	Event times not bar-aligned

Worked example (dev)

```
getManager()?.scrollToTime(barTime);
```

See also

- [Indicators](#)
- [Results and strategy](#)
- [End-user drawings](#)



EDITOR

AXIS Pine editor: CodeMirror 6, tabbed docs, docked vs popout modes, and the editor bridge.

EDITOR

The editor is the **source axis of intent**: Pine text the operator authorizes the engine to run. It is not a language server host by default in the PWA path (LSP lives in the broader pynescrypt project).

Abstract

Module	Role
<code>EditorPane.tsx</code>	Docked/standalone chrome, resize, detach
<code>tabbed-editor.tsx</code>	Multi-tab document UX
<code>PineEditor.tsx</code>	CM6 mount
<code>pine-language.ts</code>	StreamLanguage / highlighting
<code>editor-bridge.ts</code>	Cross-window doc + run messages
<code>cm-void.ts</code>	Void-theme CM styling

Conceptual model

```
sequenceDiagram
    participant Main as Main AXIS
    participant Bridge as editor-bridge
    participant Pop as Popout window
    Main->>Bridge: writeSharedDoc
    Main->>Pop: openEditorWindow
    Pop->>Bridge: hello role=editor
    Main->>Bridge: doc push
    Pop->>Bridge: run
    Bridge->>Main: runAndApply on main
    Main->>Bridge: run-status
```

Why run on main? Chart, bars, and pane manager live in the main window. Popout is a pure editing surface.

Interface surface

Docked mode

- Right column in `app.tsx` when `editor.open && mode === 'docked'`
- Resize via `ResizeHandle` → `editor.width`
- Hide chevron → `setEditorOpen(false)`
- Detach → popup or tab (`openEditorWindow`)



Popout mode

- Main shows “Editor detached” chip + **Reattach**
- Shared doc key + bridge messages: `hello`, `doc`, `run`, `run-status`, `reattach`, `popout-opened`, `popout-closed`
- Stale popout mode after reload: main waits for editor hello

Run entry points

- Editor Run control → `onRun(doc)` → `runAndApply`
- Topbar Run uses same `editorRef.getDoc()`
- Popout Run → bridge → main executes

Library load

`loadLibraryDoc(doc, name)` (when provided on ref) injects storage reads without fighting CM transaction history poorly—panel calls through App wiring.

Internals

editorRef pattern

App holds a mutable ref object `{ getDoc, setDoc?, loadLibraryDoc? }` populated by child mount—avoids over-reactive store writes on every keystroke for the full document (doc also optionally mirrored to `EDITOR_DOC_KEY`).

Language package

`pine-language.ts` supplies CM language support sufficient for editing comfort; **semantic diagnostics** are not a substitute for engine errors (Results / status).

Persistence

- Layout: `store.editor` in `pynescrypt.axis.v1`
- Text: `pynescrypt.axis.editor.doc` (+ legacy migration)

Invariants

1. Empty doc Run is no-op at topbar/editor handlers.
2. Popout does not own PaneManager.
3. Reattach restores shared doc into docked CM.
4. AXIS ≠ engine: editor never evaluates Pine itself.

Failure modes

Symptom	Mitigation
Two editors diverge	Prefer bridge doc events; reattach
Detached but no window	Popup blocked; allow popups; reattach
Lost text on crash	editor.doc localStorage; library Save



See also

- [UI shell](#)
- [Indicators](#)
- [Script library](#)



UI SHELL

AXIS chrome: topbar, watchlist, status bar, settings, plugin manager, logs, and layout chrome around the chart.

UI SHELL

The shell is the non-canvas chrome: navigation of axes, market selection, modals, and status. It binds human gestures to store + plugins without owning calculation.

Abstract

Component	Path	Role
Topbar	ui/Topbar.tsx	Axes pickers, Load/Run/Live, theme, modals
Watchlist	ui/Watchlist.tsx	Symbols, quotes, click-to-load
StatusBar	ui/StatusBar.tsx	Status, engine/storage badges, trade snippet
SettingsDialog	ui/SettingsDialog.tsx	Endpoint, engine, storage, intervals
PluginManager	ui/PluginManager.tsx	Catalog / install / library
SystemLogs	ui/SystemLogs.tsx	Ring buffer of <code>store.logs</code>
ResultsPanel	ui/ResultsPanel.tsx	Bottom drawer (see results doc)
Icons	ui/icons.tsx	Lucide wrappers
ResizeHandle	ui/ResizeHandle.tsx	Panel widths/heights

Conceptual model

```
flowchart LR
  TOP[Topbar] --> AP[setActivePlugin]
  TOP --> LOAD[loadSymbolData]
  TOP --> RUN[runAndApply]
  TOP --> LIVE[startLive/stopLive]
  WL[Watchlist] --> LOAD
  SET[Settings] --> AP
  PM[PluginManager] --> REG[registry]
  PM --> LIB[storage service]
  SB[StatusBar] --> ST[store.status]
```



Topbar responsibilities

Control	Effect
Symbol / interval	Store + Load
Source select	<code>setActivePlugin('source')</code> + default stream
Stream select	<code>live.streamId</code> / active stream
Engine select	Active engine (+ pyodide preload side effects)
CSV file	<code>parse0h1cvFile</code> → upload-store → load
Load	Historical fetch
Run	Editor doc → <code>runAndApply</code>
Live	multiplex start/stop
Editor / watchlist toggles	Layout flags
Settings / Plugins	Modal open
Detach editor	Bridge + popout

`catalogTick` forces memo re-list when plugins install/remove.

Watchlist

- Default universe includes majors (`BTCUSDT` , ...)
- `refreshSec` clamped 5–120
- Source-aware labels (e.g. CSV mode)
- Click sets symbol and loads through active source

Status bar

Single row:

- **Left — Connection HUD** (`ConnectionHud` , `data-testid="axis-connection-hud"`)
 - LIVE badge (off / connecting / live / err)
 - Tick pulse (fixed-width price + age; ping does not resize layout)
 - Engine chip (id · interpret/compile · latency)
 - Plane chips SRC / STR / ENG / STO with transport badges **WS / REST / LOCAL / BROKER**
 - Source ↔ stream pairing warning when mismatched
- **Right —** `status` / `statusMessage` , strategy snippet, log toggle, bar/ind/pane counts

Telemetry is ephemeral (`store.telemetry`); HUD compact mode and live prefs persist via settings.



Live settings (Settings dialog)

Setting	Default	Effect
Auto-start live after Load	off	<code>live.preferAfterLoad</code> → multiplex after successful Load
Indicator re-run	every-tick	or <code>bar-close</code> only
Compact connection HUD	off	hide plane chips

Logs

- `appendLog(level, message, source)`
- Cap `MAX_LOGS` (500)
- Not persisted
- Boot messages, pyodide ready, live errors

Theming

`store.theme` → `document.documentElement data-theme` (`dark` | `light`). Void dark is default product aesthetic.

Accessibility / test hooks

E2E selectors use `data-testid` on critical controls (`axis-btn-load`, `axis-manager`, ...) per `TESTING.md`.

Invariants

1. Shell never calls Python.
2. Plugin lists always come from registry facades (`listSources`, ...).
3. Modal settings persist only on Save (not on every keystroke of endpoint field until save).
4. Live off leaves historical bars intact.

Failure modes

Symptom	Shell-side check
Pickers empty	Builtins not registered at boot
Live ignores click	Stream <code>none</code> or multiplex error in logs
Settings probe fails	Network / CORS—not topbar bug

See also

- [Manager and settings](#)
- [Store](#)
- [Architecture overview](#)



INDICATORS

AXIS indicator UI: runAndApply pipeline, active engine binding, overlay routing, and IndicatorPanel.

INDICATORS

“Indicators” in AXIS terms means **scripts applied to the chart**, whether `indicator()` or `strategy()` in Pine. The module boundary is `indicators/runner.ts` plus list UI—not the PYNE evaluator.

Abstract

Piece	Role
<code>runScript</code>	Engine invocation + status/timeouts
<code>runAndApply</code>	Chart + lastRun + results open
<code>probeEndpoint</code>	Settings health check helper
<code>IndicatorPanel</code>	List/toggle scripts in store
<code>IndicatorCard</code>	Per-script UI chrome

Language semantics: [PYNE runtime](#).

Conceptual model

```
flowchart TB
  DOC[Pine source] --> RS[runScript]
  BARS[store.bars] --> RS
  CFG[getActiveEngineConfig] --> RS
  RS --> ENG[EnginePlugin.run]
  ENG --> RR[RunResult]
  RR --> LR[setLastRun]
  RR --> CH[PaneManager overlays]
  RR --> MK[Trade markers]
  RR --> DR[Script drawings]
  RR --> RES[Results open]
```

Interface surface

runScript options

Option	Default	Meaning
<code>silent</code>	false	Quiet status; live re-run logging
<code>timeout</code>	derived	Interactive: scales with bars (60–180s); silent 45s



runAndApply options

Option	Default	Meaning
<code>openResults</code>	<code>!silent</code>	Open results drawer
<code>indicatorId</code>	optional	Update existing store indicator

Overlay routing

```
overlay = result.meta.overlay !== false
paneId  = overlay ? 'price' : 'indicator'
```

Non-overlay creates indicator pane if missing (`addPane` + manager `createPane`).

Series selection

Prefer `result.series` entries whose keys do not start with `_` / `__` . Fallback: single `plots[]` array as one line named from `script_name` .

Colors: `plot_meta[k].color` or `PLOT_PALETTE[i]` .

Indicator panel

Side list of `store.scripts` (`Indicator` : id, name, code, paneId, visible, plots). Visibility toggles affect display intent; re-run still engine-driven.

Active engine binding

```
// plugins/active.ts pattern
getActiveEngine()      // registry lookup by activePlugins.engine
getActiveEngineConfig() // pluginsConfig + store.endpoint merge
```

Switching engines mid-session does not auto-rerun; operator hits Run.

Live re-run coupling

Streams may set `live.needsRerun` ; multiplex/runner path can call `runAndApply` with `silent: true` so the status bar is not spammed. Failures go to logs.

Internals

Path	Role
<code>frontend/src/indicators/runner.ts</code>	Pipeline
<code>frontend/src/indicators/IndicatorPanel.tsx</code>	List UI
<code>frontend/src/plugins/active.ts</code>	Active engine/source helpers
<code>frontend/src/engines/catalog.ts</code>	Built-in engines



Invariants

- 1. Runner is the **only** chart-facing calc orchestrator (avoid duplicate apply logic in components).
- 2. Errors still `setLastRun` with `status: 'error'` when possible.
- 3. Clearing overlays happens before re-adding—no unbounded series leak.
- 4. AXIS does not interpret Pine; it only maps `RunResult` .

Failure modes

Symptom	Layer
Engine exception	Results error + status
Success empty plots	Script/meta—not runner
Manager null	Chart not mounted; run still sets lastRun
AbortError	Timeout—reduce bars or optimize script

See also

- [Charting](#)
- [Results and strategy](#)
- [ADR-002 / ADR-004](#)



RESULTS AND STRATEGY

UI-side event normalization, trade pairing, metrics, equity curve, and ResultsPanel export pipeline.

RESULTS AND STRATEGY

Results UI turns opaque engine payloads into **operator-consumable tables, metrics, and files**. It is a pure-ish viewer layer over `lastRun` + `bars`.

Abstract

Module	Role
<code>results/events.ts</code>	Normalize heterogeneous event shapes; markers; equity series
<code>results/strategy.ts</code>	Pair trades; stats; CSV helpers
<code>ui/ResultsPanel.tsx</code>	Tabs, export, jump-to-time
<code>ui/StatusBar.tsx</code>	Compact trade summary

Conceptual model

```
flowchart LR
  LR[lastRun.events] --> N[normalizeStrategyEvents]
  B[store.bars] --> N
  N --> T[buildStrategyReport]
  N --> M[eventsToMarkers]
  N --> E[buildEquityCurve]
  T --> UI[Strategy tab]
  LR --> P[Plots / Metrics / Raw]
  T --> CSV[trades CSV]
  LR --> JSON[run JSON]
```

Event normalization

Engines may emit Pro API parity fields (`kind` , `bar_time` , `direction` , `ohlc`) or legacy UI fields (`type` , `time` , `dir` , `price`). Normalizer:

- Unifies time/price/dir/id
- Optionally fills price from bars when OHLC present/empty
- Can include order-like events for the Events tab

Trade pairing algorithm

Sketch (`buildStrategyReport`):

1. Sort by time.
2. On entry-like kind → open map by `id` .
3. On exit/close-like → match id, else sole open trade.



4. $PnL = \Delta price \times \text{long/short sign}$.

5. Aggregate win rate, profit factor, max DD on cumulative PnL.

Not modeled: fees, funding, partial fills, portfolio multi-symbol—viewer honesty.

ResultsPanel tabs

Tab	Data source
Events	<code>normalizedEvents</code>
Strategy	<code>report.trades</code> + <code>report.stats</code>
Plots	<code>series</code> / <code>plots</code> summary
Metrics	meta + context + stats subset
Raw	<code>JSON.stringify(lastRun)</code>

Interactions: copy, export, `scrollToTime` on trade rows.

Equity

`buildEquityCurve` feeds chart equity pane when runner/strategy path requests it—derived series, not engine-native unless also provided.

Export (ADR-013)

Artifact	Function
<code>axis-run-*.json</code>	Full payload
<code>axis-trades-*.csv</code>	<code>tradesToCsv</code>
Clipboard	CSV when permitted

Invariants

- Results recompute from `lastRun` reactively; they do not re-query engines.
- Changing bars without re-run can change normalization fills—re-run after Load.
- `lastRun` not rehydrated from localStorage (session artifact).
- Infinite profit factor when no losses but profits exist—display must tolerate non-finite.

Failure modes

Symptom	AXIS explanation
Events > 0, trades = 0	Unpaired or missing prices
CSV disabled	No closed trades
Jump no-ops	Manager unmounted / invalid time



Internals cross-links

Operator narrative: [Strategy and results](#).

Architecture: [ADR-013](#).

See also

- [Indicators](#)
- [Charting](#)



STORE

AXIS Solid store: AppState shape, activePlugins, persistence rules, logging, and alignment helpers.

STORE

The Solid store is the **control-plane nexus** of AXIS. Plugins read configuration from it; UI binds reactively; persistence snapshots a safe subset to `pynescrypt.axis.v1`.

Abstract

Export	Role
<code>store / setStore</code>	<code>createStore<AppState></code>
<code>persist</code>	Debounced localStorage write
<code>setActivePlugin</code>	Kinded selection + flat field mirrors
<code>appendLog</code> / status helpers	Operator-visible telemetry
<code>STORAGE_KEY</code>	<code>pynescrypt.axis.v1</code>

Types: `frontend/src/store/types.ts`.

Conceptual model

```
flowchart TB
    UI[UI Solid components] -->|setStore| S[store]
    S -->|persist subset| LS[(localStorage)]
    LS -->|loadPersisted| S
    S --> ENG[engine config readers]
    S --> SRC[source/stream readers]
    S --> CH[chart effects]
```



AppState map

Field	Persistence	Notes
bars	no	Session history
symbol , interval , exchange	yes	Market context
source , engine	yes	Flat mirrors
endpoint	yes	Server engine base URL
activePlugins	yes	Canonical four-tuple
pluginsConfig	yes	Per-plugin fields
scripts , panes	yes	Indicator list + layout
live	partial	streamId important
theme	yes	dark/light
editor , watchlist , panels	yes	Chrome geometry
stream.status	yes/volatile	Connection hint
status , statusMessage , lastRunMs	yes*	UX; ok to restore
lastRun	no	Forced null on hydrate
logs	no	Forced empty
drawingTool	reset cursor	On hydrate
drawings	yes	User annotations

*Status fields may restore but boot also appends fresh logs.

Defaults (product)

Representative defaults from store:

Key	Default
symbol	BTCUSDT
interval	1d
source / active source	binance-rest
stream	binance-ws
engine	server
storage	local
endpoint	demo/API host or localhost depending on build era
editor width	460 docked open
watchlist	majors, 15s refresh

Treat demo endpoints as **overridable**—operators should set local URLs in desk topology.



setActivePlugin

```
setActivePlugin(kind, id):
  activePlugins[kind] = id
  if source → source = id
  if engine → engine = id
  if stream → live.streamId = id
  persist()
```

Prevents registry/store split brain ([architecture overview](#)).

Logging

```
appendLog(level, message, source?)
levels: info | ok | warn | error
MAX_LOGS = 500
```

`setStatus` maps app status to log levels for important transitions.

Legacy dual path

Vanilla `state.js` remains for legacy shell with the same storage key family. Product AXIS uses Solid store only—do not dual-write new features to `state.js` unless maintaining legacy.

Invariants

1. Hydrate never restores `lastRun` or `logs`.
2. Persist strips `bars`.
3. Migration from SuperChart keys is read-repair ([state namespaces](#)).
4. `pluginsConfig` keys prefer ``${kind}:${id}``.
5. Store is not a plugin registry—ids must exist in registry to function at runtime.

Failure modes

Mode	Behavior
JSON parse fail	Defaults
localStorage throws	No-op persist
Unknown plugin id	UI may show id; run/load fails at <code>getActive*</code>
Oversized drawings	Quota risk—export/clear

Testing notes

- Import `tests/setup.ts` for localStorage stubs
- Unit coverage includes store + migration patterns
- Do not require real IDB in pure store tests



See also

- [State namespaces](#)
- [ADR-011](#)
- [UI shell](#)